



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Data & Messaging: 3.5 → 4.0

Nine changes across your stores and streams

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams THIS GUIDE
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

9 CHANGES

3 × TIER 1 - WON'T BUILD

3 × TIER 2 - WON'T START

3 × TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
1	Elasticsearch RestClient Renamed to Rest5Client	S	•
1	Kafka StreamsBuilderFactoryBeanCustomizer Removed	S	•
1	Spring AMQP RabbitRetryTemplateCustomizer Removed	S	•
2	Spring Pulsar Reactive Auto-Configuration Removed	M	•
2	Spring Session Hazelcast Auto-Configuration Removed	M	•
2	Spring Session MongoDB Auto-Configuration Removed	M	•
3	MongoDB Configuration Properties Renamed	S	Y
3	MongoDB UUID and BigDecimal Representations No Longer Defaulted	M	•
3	Spring Session Property Prefixes Renamed	S	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

MongoDB Configuration Properties Renamed

THE HUMAN LIST

Read the card. Check your code. Decide.

Elasticsearch RestClient Renamed to Rest5Client

Kafka StreamsBuilderFactoryBeanCustomizer Removed

Spring AMQP RabbitRetryTemplateCustomizer Removed

Spring Pulsar Reactive Auto-Configuration Removed

Spring Session Hazelcast Auto-Configuration Removed

Spring Session MongoDB Auto-Configuration Removed

MongoDB UUID and BigDecimal Representations No Longer Defaulted

Spring Session Property Prefixes Renamed

TIER 1

Tier 1 - Won't Build

3 cards

1. Elasticsearch RestClient Renamed to Rest5Client
2. Kafka StreamsBuilderFactoryBeanCustomizer
Removed

3. Spring AMQP RabbitRetryTemplateCustomizer
Removed

Elasticsearch RestClient Renamed to Rest5Client

Tier 1 - Won't Build

Data & Messaging

Impact: S

Spring Boot 4.0 auto-configures Rest5Client instead of RestClient for Elasticsearch. Code injecting the old type won't compile.

WHAT CHANGED

Spring Boot 4.0 auto-configures `org.elasticsearch.client.Rest5Client` instead of `org.elasticsearch.client.RestClient`. Components that inject `RestClient` directly will fail because the auto-configured bean is now of type `Rest5Client`.

WHY

The Elasticsearch Java client library introduced `Rest5Client` as the successor to the original `RestClient`. Spring Boot 4.0 adopted the newer type to track the upstream library.

SCOPE CHECK

Search for `org.elasticsearch.client.RestClient` and `RestClientBuilderCustomizer` in your Java sources. Any direct injection of the low-level client is affected.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/ElasticsearchUsage.java:[5,35]
error: cannot find symbol
  symbol:   class RestClient
  location: package org.elasticsearch.client
```

THE FIX

Import

BEFORE

```
import org.elasticsearch.client.RestClient;
```

AFTER

```
import org.elasticsearch.client.Rest5Client;
```

Injection

BEFORE

```
@Autowired
private RestClient restClient;
```

AFTER

```
@Autowired
private Rest5Client restClient;
```

Customizer bean (if used)

BEFORE

```
@Bean
public RestClientBuilderCustomizer customizer() { ... }
```

AFTER

```
@Bean
public Rest5ClientBuilderCustomizer customizer() { ... }
```

HOW TO FIX

Rename RestClient to Rest5Client.

Replace `RestClient` with `Rest5Client` in all injection points, customiser beans, and direct usages. The API is broadly compatible, but check the Elasticsearch client changelog for subtle differences.

WATCH OUT

Do not confuse Elasticsearch's `RestClient` with Spring Framework's `org.springframework.web.client.RestClient`; they are different classes, and only the Elasticsearch one changed.

FURTHER INFO

The high-level `RestHighLevelClient` was already removed in Spring Boot 3.x; this is the next step in the same direction.

spring-break module: `elasticsearch-rest5client` Spring Boot 4.0 Migration Guide

Kafka StreamsBuilderFactoryBeanCustomizer Removed

Tier 1 - Won't Build

Data & Messaging

Impact: S

Spring Boot's StreamsBuilderFactoryBeanCustomizer is gone. Replace it with Spring Kafka's StreamsBuilderFactoryBeanConfigurer.

WHAT CHANGED

`org.springframework.boot.autoconfigure.kafka.StreamsBuilderFactoryBeanCustomizer` has been removed. The replacement is `org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer` from Spring Kafka itself.

WHY

Spring Boot's interface was a thin duplicate of one Spring Kafka already provides. Removing it cuts an unnecessary indirection.

SCOPE CHECK

Search for `StreamsBuilderFactoryBeanCustomizer` in all Java/Kotlin sources. Only Kafka Streams applications are affected.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/KafkaCustomizerUsage.java:[3,60]
  error: package org.springframework.boot.autoconfigure.kafka does not contain
    StreamsBuilderFactoryBeanCustomizer
```

THE FIX

Import

BEFORE

```
import org.springframework.boot.autoconfigure.kafka.StreamsBuilderFactoryBeanCustomizer;
```

AFTER

```
import org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer;
```

Bean method signature

BEFORE

```
@Bean
public StreamsBuilderFactoryBeanCustomizer customizer() { ... }
```

AFTER

```
@Bean
public StreamsBuilderFactoryBeanConfigurer configurer() { ... }
```

implements clause

BEFORE

```
public class MyCustomizer implements StreamBuilderFactoryBeanCustomizer {
```

AFTER

```
public class MyCustomizer implements StreamBuilderFactoryBeanConfigurer {
```

HOW TO FIX

Switch to StreamBuilderFactoryBeanConfigurer.

Replace the import and the implements / return type declaration. The interface method signature is compatible: the method you override is `configure(StreamBuilderFactoryBean)` in both cases.

WATCH OUT

The method name on

`StreamBuilderFactoryBeanConfigurer` is `configure`, same as the old `customize`. Verify the exact method name in the Spring Kafka version you're using: it differs between minor versions.

FURTHER INFO

The removal is part of shrinking Boot's Kafka auto-configuration surface; the upstream Spring Kafka interface is the long-term home.

spring-break module: kafka-streams-customizer-removed [Spring Boot 4.0 Migration Guide](#)

Spring AMQP RabbitRetryTemplateCustomizer Removed

Tier 1 - Won't Build

Data & Messaging

Impact: S

Boot 4.0 removed `RabbitRetryTemplateCustomizer`. Every bean implementing it fails to compile; publisher and consumer retry now have separate customiser interfaces.

WHAT CHANGED

Spring AMQP 4.0 dropped its dependency on Spring Retry and now uses Spring Framework's retry abstraction.

`RabbitRetryTemplateCustomizer` is replaced by `RabbitTemplateRetrySettingsCustomizer` (publisher-side) and `RabbitListenerRetrySettingsCustomizer` (consumer-side).

WHY

Spring Boot 4.0 dropped Spring Retry across the board in favour of Spring Framework's native retry support. Spring AMQP followed the same path.

SCOPE CHECK

Grep for

`RabbitRetryTemplateCustomizer` across your source tree. Also check `import org.springframework.amqp.rabbit.retry` imports.

WHAT YOU'LL SEE

```
@Bean
public RabbitRetryTemplateCustomizer retryCustomizer() {
    return (target, retryTemplate) -> { ... };
}

// Boot 4.0 compile error:
error: cannot find symbol
    symbol: class RabbitRetryTemplateCustomizer
```

THE FIX

Publisher-side retry customizer

BEFORE

```
import org.springframework.amqp.rabbit.retry.RabbitRetryTemplateCustomizer;

@Bean
public RabbitRetryTemplateCustomizer retryCustomizer() {
    return (target, retryTemplate) -> {
        retryTemplate.setRetryPolicy(...);
    };
}
```

AFTER

```
import org.springframework.boot.autoconfigure.amqp.RabbitTemplateRetrySettingsCustomizer;

@Bean
public RabbitTemplateRetrySettingsCustomizer retryCustomizer() {
    return settings -> {
        settings.setMaxAttempts(3);
    };
}
```

Consumer-side retry customizer

BEFORE

```
// Previously both publisher and consumer shared RabbitRetryTemplateCustomizer
```

AFTER

```
import org.springframework.boot.autoconfigure.amqp.RabbitListenerRetrySettingsCustomizer;

@Bean
public RabbitListenerRetrySettingsCustomizer listenerRetryCustomizer() {
    return settings -> {
        settings.setMaxAttempts(5);
    };
}
```

HOW TO FIX

Replace with the split customiser interfaces.

Replace each `RabbitRetryTemplateCustomizer` bean with the publisher or consumer variant, whichever side it configured. The new interfaces set retry settings directly rather than manipulating a `RetryTemplate`.

WATCH OUT

The new customiser interfaces accept a settings object rather than a `RetryTemplate`. Retry policy logic expressed as `RetryTemplate` configuration must be re-expressed using the properties on the settings object.

FURTHER INFO

One of several retry-related breaks in Boot 4.0. See also: [spring-retry-removed](#), [retry-semantic-change](#).

spring-break module: [spring-amqp-retry-removed](#) [Spring Boot 4.0 Migration Guide](#)

TIER 2

Tier 2 - Won't Start

3 cards

1. Spring Pulsar Reactive Auto-Configuration
Removed
2. Spring Session Hazelcast Auto-Configuration
Removed

3. Spring Session MongoDB Auto-Configuration
Removed

Spring Pulsar Reactive Auto-Configuration Removed

Tier 2 - Won't Start

Data & Messaging

Impact: M

Spring Boot 4.0 dropped reactive Pulsar auto-configuration.

`ReactivePulsarTemplate` and `ReactivePulsarClient` beans are no longer available. The app starts but fails on first use.

WHAT CHANGED

`spring-boot-starter-pulsar-reactive` and the corresponding auto-configuration module were removed from Spring Boot 4.0. The auto-configuration for `ReactivePulsarClient`, `ReactivePulsarTemplate`, and `ReactiveMessageListenerContainer` no longer exists in Boot's auto-configure artifact.

WHY

Boot 4.0's modular restructuring dropped auto-configurations for reactive integration layers with limited adoption relative to their maintenance cost. The imperative Spring Pulsar integration remains fully supported.

SCOPE CHECK

Grep for `ReactivePulsarTemplate`, `ReactivePulsarClient`, and `spring-boot-starter-pulsar-reactive` across your source and build files.

WHAT YOU'LL SEE

```
// Boot 4.0 startup or runtime failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
    No qualifying bean of type
    'org.springframework.pulsar.reactive.core.ReactivePulsarTemplate' available

// Or auto-wiring failure:
Field reactivePulsarTemplate required a bean of type
    'ReactivePulsarTemplate' that could not be found.
```

THE FIX

Replace reactive auto-config with explicit bean definitions

BEFORE

```
<!-- Boot 3.5: auto-configured reactive Pulsar client -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-pulsar-reactive</artifactId>
</dependency>
```

AFTER

```
<!-- Boot 4.0: use imperative starter or configure reactive beans manually -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-pulsar</artifactId>
</dependency>
```

HOW TO FIX

Switch to the imperative Pulsar integration or configure reactive beans manually.

If the reactive API is required, declare the reactive Pulsar beans explicitly in a `@Configuration` class. If reactive messaging is not critical, switch to the imperative `spring-boot-starter-pulsar` which remains auto-configured in Boot 4.0.

VERIFY

Reactive Pulsar producers and consumers initialise correctly after removing the dependency on Boot's auto-configuration

WATCH OUT

The imperative `PulsarTemplate` and `PulsarClient` auto-configuration in `spring-boot-starter-pulsar` is unaffected by this change.

FURTHER INFO

The Spring Pulsar reactive library itself survives; only Boot's auto-configuration went away, so the beans must be declared by hand.

[spring-break module: pulsar-reactive-removed](#)

[Spring Boot 4.0 Migration Guide](#)

[Spring Pulsar Documentation](#)

Spring Session Hazelcast Auto-Configuration Removed

Tier 2 - Won't Start

Data & Messaging

Impact: M

Spring Boot 4.0 dropped built-in Hazelcast session auto-configuration. The app compiles but fails at startup with `NoSuchBeanDefinitionException`.

WHAT CHANGED

Spring Boot's auto-configuration for Hazelcast-backed Spring Session was removed from `spring-boot-autoconfigure`. The classes `HazelcastSessionConfiguration` and `HazelcastSessionProperties` no longer exist in the Spring Boot artifact. The Hazelcast team took ownership of the integration.

WHY

Boot 4.0 handed several Spring Session store integrations to the teams closest to the technology. Hazelcast and MongoDB now maintain their own, keeping Boot lean and letting each integration track its store's release cycle.

SCOPE CHECK

Check `spring.session.store-type=hazelcast` in your properties. Grep for `HazelcastIndexedSessionRepository` and `HazelcastSessionConfiguration` in your source.

WHAT YOU'LL SEE

```
// Boot 4.0 startup failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
    No qualifying bean of type
    'org.springframework.session.SessionRepository' available

// Or context load failure mentioning missing HazelcastIndexedSessionRepository
```

THE FIX

pom.xml — add Hazelcast team's Spring Session integration

BEFORE

```
<!-- Boot 3.5: built-in, no extra dependency needed beyond spring-session-hazelcast -->
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-hazelcast</artifactId>
</dependency>
```

AFTER

```
<!-- Boot 4.0: use Hazelcast team's own integration artifact -->
<dependency>
  <groupId>com.hazelcast</groupId>
  <artifactId>hazelcast-spring-session</artifactId>
  <version><!-- check Hazelcast release --></version>
</dependency>
```

HOW TO FIX

Migrate to the Hazelcast-maintained Spring Session integration.

Replace `spring-session-hazelcast` with the artifact published by the Hazelcast team and configure it from Hazelcast's own documentation.

VERIFY

Hazelcast-backed session store initialises correctly after migrating to the Hazelcast-team-maintained integration

WATCH OUT

Property keys and bean names may differ in the Hazelcast-maintained integration. Verify them against Hazelcast's own documentation rather than the Spring Boot migration guide.

FURTHER INFO

In Boot 3.5, `spring-session-hazelcast` on the classpath plus `spring.session.store-type=hazelcast` was enough: Boot wired the session store for you. That wiring is what disappeared.

spring-break module: `spring-session-hazelcast-removed` [Spring Boot 4.0 Migration Guide](#)

[Hazelcast Spring Session Documentation](#)

Spring Session MongoDB Auto-Configuration Removed

Tier 2 - Won't Start

Data & Messaging

Impact: M

Spring Boot 4.0 dropped built-in MongoDB session auto-configuration. The app compiles but fails at startup with `NoSuchBeanDefinitionException`.

WHAT CHANGED

Spring Boot's auto-configuration for MongoDB-backed Spring Session was removed from `spring-boot-autoconfigure`. The `MongoSessionConfiguration` and related auto-configuration classes no longer exist in the Spring Boot artifact. Spring Data MongoDB took ownership of the integration.

WHY

Boot 4.0 handed several Spring Session store integrations to the teams closest to the technology. MongoDB and Hazelcast now maintain their own, keeping Boot lean and letting each integration track its store's release cycle.

SCOPE CHECK

Check `spring.session.store-type=mongodb` in your properties. Grep for `MongoIndexedSessionRepository` and `MongoSessionConfiguration` in your source.

WHAT YOU'LL SEE

```
// Boot 4.0 startup failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
No qualifying bean of type
'org.springframework.session.SessionRepository' available

// Or context load failure mentioning missing MongoIndexedSessionRepository
```

THE FIX

pom.xml — switch to Spring Data MongoDB's own session support

BEFORE

```
<!-- Boot 3.5: built-in auto-config via spring-session-data-mongodb -->
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-data-mongodb</artifactId>
</dependency>
```

AFTER

```
<!-- Boot 4.0: session support is part of spring-boot-starter-data-mongodb -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
<!-- Refer to Spring Data MongoDB docs for explicit session config -->
```

HOW TO FIX

Migrate to the Spring Data MongoDB session integration.

The MongoDB team's Spring Session support is bundled with Spring Data MongoDB. Add `spring-boot-starter-data-mongodb` and configure the session repository according to Spring Data MongoDB's

documentation. The `spring.session.store-type=mongodb` property alone is no longer sufficient.

VERIFY

MongoDB-backed session store initialises correctly after migrating to the MongoDB-team-maintained integration

WATCH OUT

MongoDB session property keys changed:

`spring.session.mongodb.` *became*
`spring.session.data.mongodb.` . This is a separate
but related break covered in the session property
renames card.

FURTHER INFO

In Boot 3.5, `spring-session-data-mongodb` on the classpath plus `spring.session.store-type=mongodb` was enough: Boot wired the session repository for you. That wiring is what disappeared; the integration now ships as part of Spring Data MongoDB.

spring-break module: [spring-session-mongodb-removed](#) [Spring Boot 4.0 Migration Guide](#)

TIER 3

Tier 3 - Behaves Differently

3 cards

1. MongoDB Configuration Properties Renamed
2. MongoDB UUID and BigDecimal Representations
No Longer Defaulted
3. Spring Session Property Prefixes Renamed

MongoDB Configuration Properties Renamed

Tier 3 - Behaves Differently

Data & Messaging

Impact: S

🔗 OpenRewrite

`spring.data.mongodb.` properties are silently ignored in Boot 4.0. The new prefix is `spring.mongodb.`. The app starts but connects with defaults: wrong host, no auth, wrong database.

WHAT CHANGED

All MongoDB connection and configuration properties moved from the `spring.data.mongodb` namespace to `spring.mongodb`. Examples:

```
spring.data.mongodb.uri →  
spring.mongodb.uri ,  
spring.data.mongodb.database  
→ spring.mongodb.database .
```

Actuator management properties changed from

```
management.health.mongo. to  
management.health.mongodb. .
```

WHY

MongoDB auto-configuration moved into a dedicated persistence module, and the old prefix was a legacy of living inside Spring Data's namespace. The rename matches Boot's own namespace conventions.

SCOPE CHECK

Grep for `spring.data.mongodb` across all property files, YAML files, and `@value` annotations. Also check `management.health.mongo`.

WHAT YOU'LL SEE

```
# application.properties (Boot 3.5 style, ignored on 4.0):  
spring.data.mongodb.uri=mongodb://user:pass@prod-host:27017/mydb  
spring.data.mongodb.database=mydb  
  
# Boot 4.0: app starts, connects to localhost:27017 with no auth.  
# No error: wrong database, wrong host. Data operations succeed  
# against the wrong instance.
```

THE FIX

application.properties

BEFORE

```
spring.data.mongodb.uri=mongodb://user:pass@host:27017/mydb  
spring.data.mongodb.database=mydb  
spring.data.mongodb.auto-index-creation=true
```

AFTER

```
spring.mongodb.uri=mongodb://user:pass@host:27017/mydb  
spring.mongodb.database=mydb  
spring.mongodb.auto-index-creation=true
```

Actuator health properties

BEFORE

```
management.health.mongo.enabled=false
```

AFTER

```
management.health.mongodb.enabled=false
```

HOW TO FIX

Rename all `spring.data.mongodb.*` properties to `spring.mongodb.*`

Do a project-wide search-and-replace: `spring.data.mongodb.` → `spring.mongodb.`. Check all `application.properties`, `application.yml`, and any profile-specific variants. OpenRewrite has a migration recipe that handles this automatically.

VERIFY

Application connects to MongoDB and all configuration (URI, auth, pool size, etc.) applies correctly after renaming properties

WATCH OUT

The app starts, connects with driver defaults, and runs normally against the wrong instance. Confirm the property is being read by testing with a URI that cannot resolve, such as a non-existent host.

Spring Session MongoDB property keys changed separately: `spring.session.mongodb.` → `spring.session.data.mongodb.`. Check the [spring-session-property-renames](#) card.

FURTHER INFO

Every property under the old prefix moved, pool size and auth settings included.

spring-break module: [mongodb-property-renames](#) [Spring Boot 4.0 Migration Guide](#)

MongoDB UUID and BigDecimal Representations No Longer Defaulted

Tier 3 - Behaves Differently

Data & Messaging

Impact: M

Boot 4.0 removes the default UUID and BigDecimal BSON representations. Documents written under Boot 3.5 defaults may deserialise wrongly or fail. Configure the representations explicitly.

WHAT CHANGED

The `spring.mongodb.representation.uuid` and `spring.data.mongodb.representation.big-decimal` properties must now be configured explicitly. Spring Boot 4.0 no longer injects defaults for these representations. For compatibility with existing data, declare the same values Boot 3.5 set automatically.

WHY

Boot's defaults overrode the MongoDB driver defaults, surprising teams using MongoDB outside Spring Boot or migrating data between tools. Explicit configuration puts the application in control of the data format.

SCOPE CHECK

Grep for `UUID` fields in MongoDB document classes (`@Document`). Also find any `BigDecimal` or `BigInteger` fields. These are at risk. Run a read test against your production data snapshot before deploying.

WHAT YOU'LL SEE

```
// Boot 3.5: UUID stored as STANDARD binary subtype 4
// Boot 4.0 default: UUID stored as JAVA_LEGACY binary subtype 3

// Reading a Boot 3.5 document in a Boot 4.0 app:
org.bson.codecs.configuration.CodecConfigurationException:
  No codec found for UUID with representation JAVA_LEGACY

// Or: UUID field deserialises to null or a wrong value
```

THE FIX

application.properties — restore Boot 3.5 representations

BEFORE

```
# Boot 3.5: these were set automatically (no configuration needed)
```

AFTER

```
# Boot 4.0: must be explicit to match what Boot 3.5 used
spring.mongodb.representation.uuid=standard
# For BigDecimal – check what representation your Boot 3.5 data was written with:
spring.data.mongodb.representation.big-decimal=decimal128
```

HOW TO FIX

Declare the representations that match your existing data.

Set `spring.mongodb.representation.uuid` to `standard` if your documents were written with Boot 3.5 defaults. For `BigDecimal`, check which BSON type your existing documents use (inspect with `mongosh`) and configure `spring.data.mongodb.representation.big-decimal` accordingly.

Migrate existing data if changing representations.

To adopt different representations going forward, write a migration script to convert existing documents first. Changing representation without migrating data makes old documents unreadable.

VERIFY

UUID and BigDecimal fields round-trip correctly and existing documents with the old representation are still readable after explicit configuration

WATCH OUT

Wrong-representation reads return valid-looking, wrong UUIDs. No exception, no log line. Test against a real data snapshot. Unit tests create fresh documents and never catch representation mismatches.

FURTHER INFO

Boot 3.5 set UUID representation to `STANDARD` and BigDecimal to a specific codec. With no Boot default, the representation falls back to whatever the MongoDB driver or Spring Data MongoDB defaults to. New documents may use a different binary subtype, breaking code that inspects raw BSON.

[Spring Boot 4.0 Migration Guide](#) [MongoDB UUID Representations](#)

Spring Session Property Prefixes Renamed

Tier 3 - Behaves Differently

Data & Messaging

Impact: S

`spring.session.redis.` and `spring.session.mongodb.` are silently ignored in Boot 4.0. The new prefixes are `spring.session.data.redis.` and `spring.session.data.mongodb.`.

WHAT CHANGED

Spring Session's Redis backend properties moved from `spring.session.redis.` to `spring.session.data.redis.`. MongoDB backend properties moved from `spring.session.mongodb.` to `spring.session.data.mongodb.`. This mirrors the MongoDB connection property rename pattern (`spring.data.mongodb` → `spring.mongodb`).

WHY

The `data` infix ties each session store's configuration to its Spring Data module namespace, making the relationship explicit.

SCOPE CHECK

Grep for `spring.session.redis` and `spring.session.mongodb` across all property and YAML files.

WHAT YOU'LL SEE

```
# application.properties (Boot 3.5 style, ignored on 4.0):
spring.session.redis.namespace=myapp
spring.session.redis.flush-mode=on-save
spring.session.mongodb.collection-name=sessions

# Boot 4.0: app starts, sessions stored under default namespace.
# No error. Session isolation between apps sharing the same Redis
# instance is broken.
```

THE FIX

Redis session properties

BEFORE

```
spring.session.redis.namespace=myapp:session
spring.session.redis.flush-mode=on-save
spring.session.redis.save-mode=on-set-attribute
spring.session.redis.cleanup-cron=0 * * * * *
```

AFTER

```
spring.session.data.redis.namespace=myapp:session
spring.session.data.redis.flush-mode=on-save
spring.session.data.redis.save-mode=on-set-attribute
spring.session.data.redis.cleanup-cron=0 * * * * *
```

MongoDB session properties

BEFORE

```
spring.session.mongodb.collection-name=sessions
```

AFTER

```
spring.session.data.mongodb.collection-name=sessions
```

HOW TO FIX

Rename `spring.session.redis.*` to `spring.session.data.redis.*`

Do a project-wide search-and-replace: `spring.session.redis.` → `spring.session.data.redis.` and `spring.session.mongodb.` → `spring.session.data.mongodb.` . Check all property files, YAML files, and profile variants.

VERIFY

Session store connects correctly with the renamed property prefixes: confirm sessions persist to Redis or MongoDB with the new configuration

WATCH OUT

The session store type property (`spring.session.store-type`) is unchanged. Only the store-specific configuration sub-keys are affected.

Nothing fails at startup, which makes this hard to catch in testing. A wrong namespace in Redis means sessions are stored but never found: users appear logged out immediately after login. Confirm the fix by checking session keys exist in Redis under the expected namespace.

FURTHER INFO

Old-prefix properties still parse, so the app starts cleanly. Sessions fall back to defaults (wrong namespace, wrong collection, wrong TTL) or the auto-configuration picks the first available store type.

spring-break module: [spring-session-property-renames](#) [Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki

OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4

HeroDevs NES for Spring • herodevs.com/support/spring-nes

The test suite behind this book • github.com/spoole167/spring-break